

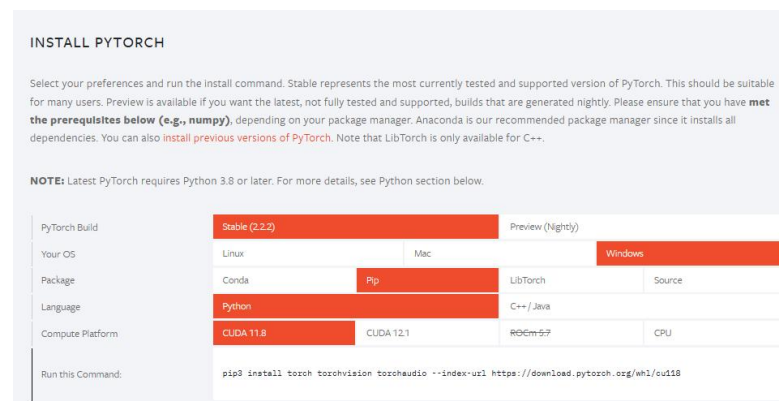
实验一：多层感知器（MLP）@ MNIST

1. 数据集：MNIST 手写数字数据集。
2. 目标：使用 PyTorch 构建并训练一个多层感知器（MLP），解决 MNIST 手写数字分类问题。
3. 关键内容：
 - 构建 MLP 模型。
 - 数据加载和预处理。
 - 训练模型并进行性能评估。

示例代码（以 pytorch 环境为例）：

步骤 1：安装 pytorch 虚拟环境

<https://pytorch.org/>



例：

如有显卡：pip3 install torch torchvision torchaudio --index-url

<https://download.pytorch.org/whl/cu118>

无显卡：pip3 install torch torchvision torchaudio --index-url

<https://download.pytorch.org/whl/cpu>

步骤 2：定义模型

我们将定义一个简单的多层感知器模型，本示例包含两个隐藏层。（各位同学根据自己模型进行调整）

示例代码：

```
1. import torch
2. import torch.nn as nn
3. import torch.nn.functional as F
4.
5. class MLP(nn.Module):
6.     def __init__(self):
7.         super(MLP, self).__init__()
8.         self.fc1 = nn.Linear(28*28, 512) # 输入层到第一个隐藏层
```

```

9.         self.fc2 = nn.Linear(512, 512)    # 第一个隐藏层到第二个隐藏层
10.        self.fc3 = nn.Linear(512, 10)     # 第二个隐藏层到输出层
11.
12.    def forward(self, x):
13.        x = x.view(-1, 28*28) # 展平输入数据
14.        x = F.relu(self.fc1(x))
15.        x = F.relu(self.fc2(x))
16.        x = self.fc3(x)
17.        return x

```

步骤 3: DataLoader

我们将使用 torchvision 来加载 MNIST 数据集，并创建训练和测试的 DataLoader。

示例代码：

```

1. from torchvision import datasets, transforms
2. from torch.utils.data import DataLoader
3.
4. # 定义预处理函数
5. transform = transforms.Compose([
6.     transforms.ToTensor(),
7.     transforms.Normalize((0.5,), (0.5,))
8. ])
9.
10. # 加载数据集
11. train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
12. test_dataset = datasets.MNIST(root='./data', train=False, download=True, transform=transform)
13.
14. # 创建数据加载器
15. train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
16. test_loader = DataLoader(test_dataset, batch_size=1000, shuffle=False)

```

步骤 4: 模型训练与验证

我们将训练模型并在测试集上评估性能，本示例使用 SGD 优化器，同学可以自行调整。

示例代码：

```

1. from torch import optim
2.
3. device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
4.
5. model = MLP().to(device)
6. optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.5)

```

```

7. criterion = nn.CrossEntropyLoss()
8.
9. def train(epoch):
10.     model.train()
11.     for batch_idx, (data, target) in enumerate(train_loader):
12.         data, target = data.to(device), target.to(device)
13.         optimizer.zero_grad()
14.         output = model(data)
15.         loss = criterion(output, target)
16.         loss.backward()
17.         optimizer.step()
18.         if batch_idx % 100 == 0:
19.             print(f"Train Epoch: {epoch} [{batch_idx * len(data)} / {len(train_loader.dataset)}] ({100. * batch_idx / len(train_loader):.0f}%) \tLoss: {loss.item():.6f}")
20.
21. def test():
22.     model.eval()
23.     test_loss = 0
24.     correct = 0
25.     with torch.no_grad():
26.         for data, target in test_loader:
27.             data, target = data.to(device), target.to(device)
28.             output = model(data)
29.             test_loss += criterion(output, target).item()
30.             pred = output.argmax(dim=1, keepdim=True)
31.             correct += pred.eq(target.view_as(pred)).sum().item()
32.     test_loss /= len(test_loader.dataset)
33.     print(f'\nTest set: Average loss: {test_loss:.4f}, Accuracy: {correct} / {len(test_loader.dataset)} ({100. * correct / len(test_loader.dataset):.0f}%) \n')
34.
35. # 训练和测试模型
36. for epoch in range(1, 11):
37.     train(epoch)
38.     test()

```

实验二：Kohonen(SOM) @ MNIST

1. 数据集：MNIST 手写数字数据集。
2. 目标：在使用非典型的神经网络模型之一 BAM（双向联想记忆）网络、Hopfield 网络或 Kohonen 自组织映射网络——来处理 MNIST 手写数字数据集。

示例代码：

以 Kohonen (SOM) 网络为例。

步骤 1：加载数据集和预处理

```
1. from torchvision.datasets import MNIST
2. from torchvision.transforms import ToTensor
3. from torch.utils.data import DataLoader
4. from minisom import MiniSom
5. import numpy as np
6.
7. # 加载训练集和测试集
8. mnist_train = MNIST("/home/zhu/data/MNIST/", download=True, transform=ToTensor(), train=True)
9. mnist_test = MNIST("/home/zhu/data/MNIST/", download=True, transform=ToTensor(), train=False)
10.
11. # 创建数据加载器
12. train_loader = DataLoader(mnist_train, batch_size=len(mnist_train), shuffle=True)
13. test_loader = DataLoader(mnist_test, batch_size=len(mnist_test), shuffle=False)
14. train_data, train_labels = next(iter(train_loader))
15. test_data, test_labels = next(iter(test_loader))
16. train_data = train_data.numpy()
17. train_labels = train_labels.numpy()
18. test_data = test_data.numpy()
19. test_labels = test_labels.numpy()
20.
21. # 数据预处理：归一化和展平
22. x_train = train_data.reshape(-1, 28*28)
23. x_test = test_data.reshape(-1, 28*28)
```

步骤 2：初始化模型训练

```
1. # 初始化 SOM
2. som = MiniSom(20, 20, 28*28, sigma=1.0, learning_rate=0.5)
3.
4. # 训练 SOM
```

```
5. print("Training SOM on training data...")
6. som.train_random(x_train, 100) # 使用 100 个训练周期
7. print("...done!")
```

步骤 3：测试模型性能

```
1. # 确定每个 SOM 节点的代表数字
2. node_labels = np.zeros((20, 20), dtype=np.int64) - 1 # 使用 -1 初始化表示节点未被标记
3. for x, label in zip(x_train, train_labels):
4.     w = som.winner(x)
5.     node_labels[w] = label if node_labels[w] == -1 else (node_labels[w] if node_labels[w] == label else -1)
6.
7. # 使用 SOM 对测试集进行预测
8. correct_predictions = 0
9. for x, label in zip(x_test, test_labels):
10.    w = som.winner(x)
11.    if node_labels[w] == label:
12.        correct_predictions += 1
13.
14. # 计算测试集准确率
15. accuracy = correct_predictions / len(test_labels)
16. print(f"Accuracy on test set: {accuracy * 100:.2f}%")
```

本示例代码使用 torch 和 minisom 实现，各位同学也可使用其他库实现。本代码仅做启发用，大家自行对网络进行实现和改进，也可与其他网络结构联合优化。